

ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky, Ilya Sutskever, Geoffrey E. Hinton

Alexnet

Used deep CNN, trained by 1.2 million images in ImageNet ILSVRC-2010 into 1000 classes.

Has 60 million parameters and 650,000 neurons (five convolutional layers, some max-pooling layers, and three fully-connected layers, lastly softmax.)

Used non-saturating neurons (Fast training), dropout (avoid overfitting)

Since large dataset isn't sufficient for recognizing realistic objects, models need a prior-knowledge (locality - near pixels have relevance, translation invariance - move of image is still the same object) to compensate it. -> **We used CNN for prior-knowledge.**

Less connection and parameters compared to FC. (Actually has slightly lower theoretically-best performance)

Section 3 -> contains new and unusual features which improve its performance and reduce its training time

Section 4 -> contains effective techniques for preventing overfitting

Seems to need faster GPUs and bigger datasets for improvement

Section 2 - DataSet :

Imagenet - Has 15 million datasets with 22000 labels

ILSVRC-2010 - 1.2 million training images, 50,000 validation images, and 150,000 testing images.

AlexNet requires fixed size of image, so downsampled to 256x256. - rescaled shorter size to 256 and cropped centered 256x256.

Section3 - Architecture : Eight learned layers — five convolutional and three fully-connected.

3.1 : Instead of using tanh or sigmoid, used RELU - Faster training since non-saturating

3.2 : Used two GPU - Only communicate on certain layers(cross-GPU parallelization), Even one GPU had more kernels respectively, two GPU have better performance

3.3 : Local Response Normalization(LRN) - create competition between neurons computed by different kernels, making strong ones stronger, weak ones weaker. -> aids generalization(일반화, 학습한 것을 새로운 데이터에 잘 적용하는 능력)

3.4 : Overlapping Pooling - on $s(\text{stride}) \times z(\text{pooling size})$, by increasing $z(s < z)$, overlapping happened on pixels(pixel이 여러 pooling 영역에 속함)

3.5 : Overall Architecture - five convolutional and three fully-connected + 1000-way softmax(1000 labels).

Only third layer is connected to two GPU.

Response-normalization(LRN) applied after Conv1, Conv2. Max-pooling applied after two LRN and Conv5

RELU on every Convolutional+FC layer.

First convolutional layer - kernels of size $11 \times 11 \times 3$ with a stride of 4 pixels(distance between field centers) -> $224 \times 224 \times 3$ to $55 \times 55 \times 48$ (CNN) to $27 \times 27 \times 48$ (Maxpooling $s=2$, $z=3$)

Second convolutional layer - kernels of size $5 \times 5 \times 48$ with stride 1 -> $27 \times 27 \times 48$ to $27 \times 27 \times 128$ (padding) to $13 \times 13 \times 128$ (maxpooling)

Third convolutional layer - 384 kernels of size $3 \times 3 \times 256$ -> $13 \times 13 \times 128$ to GPU당: $13 \times 13 \times 192$

Fourth convolutional layer - 384 kernels of size $3 \times 3 \times 192$ -> $13 \times 13 \times 192$ to $13 \times 13 \times 192$

Fifth convolutional layer - 256 kernels of size $3 \times 3 \times 192$ -> $13 \times 13 \times 192$ to $13 \times 13 \times 128$

Section 4 - Reducing Overfitting

60 million parameters -> 1000 classes impose only 10 bits of constraint per training example -> insufficient -> overfitting problem

4.1 : Data Augmentation

Method1 : 이미지 자르기 + 뒤집기

Extract random 224×224 patches + horizontal reflections(좌우반전) from 256×256 images -> training set $\times 2048$

At test time: extract 5 patches (4 corners + center) + horizontal reflections = 10 patches total -> average predictions

Method2 : RGB 색상 변형

PCA(주성분 분석) on RGB pixel values -> add multiples of principal components(noise) with random magnitudes (α_i drawn from Gaussian, mean=0, std=0.1) $\rightarrow$ color/intensity variation

Both methods computationally free (generated on CPU while GPU trains previous batch)

Method2 reduces top-1 error by over 1%

4.2 : Dropout

Sets each hidden neuron output to 0 with probability 0.5 during training(50프로 확률로 뉴런 종료)

Dropped neurons do not contribute to forward pass or backpropagation

Every input -> different architecture sampled, but all share weights

Prevents co-adaptation -> forces learning robust features useful with many different subsets of neurons

At test time: use all neurons but multiply outputs by 0.5

Used in first two FC layers

Roughly doubles training iterations to converge

Section 5 - Details of Learning

SGD(Stochastic Gradient Descent로 학습): batch size=128, momentum=0.9, weight decay=0.0005

Weight decay not just regularizer(과적합 방지) -> actually reduces training error

Weights initialized from zero-mean Gaussian (average = 0, std=0.01)

Biases in Conv2, Conv4, Conv5 and FC layers initialized to 1 -> provides positive ReLU inputs which accelerates early learning

Biases in remaining layers initialized to 0

Equal learning rate for all layers, initialized at 0.01 -> divided by 10 when validation error stops improving (decreased 3 times total)

Trained for ~90 epochs on 1.2M images -> took 5-6 days on two NVIDIA GTX 580 3GB GPUs

Section 6 - Results

ILSVRC-2010

AlexNet: top-1 37.5%, top-5 17.0%

Previous best: 47.1%, 28.2% (sparse coding)

ILSVRC-2012:

1 CNN: top-5 18.2%

5 CNNs ensemble: top-5 16.4%

7 CNNs* (pretrained on ImageNet Fall 2011): top-5 15.3%

2nd place: 26.2%

Section 7 - Discussion

Large deep CNN achieves record-breaking results using purely supervised learning

Depth is critical -> removing any middle Conv layer -> ~2% top-1 performance loss